

Programação com Sockets

Nome: Pedro Lucas Paixão Agres Carvalho

Matrícula: 222116242

Introdução

O presente trabalho busca descrever o processo, as diretivas, os resultados e uma análise destes sobre o desenvolvimento de um par de programas que operam no modelo cliente-servidor e exercitam a comunicação do tipo requisição-resposta sobre o protocolo TCP, utilizando a biblioteca de sockets do python e seguindo o paradigma da computação em nuvem. A implementação foi desenvolvida para ser compatível com os protocolos de endereçamento IPv4 e IPv6, garantindo flexibilidade e robustez na camada de rede. A função específica deste sistema é a transferência de informações de diretórios, simulando o acesso a recursos remotos.

A operação segue um padrão simples de comunicação e transferência: o cliente estabelece a conexão TCP com o servidor e envia a mensagem "READY". O servidor responde com "READY ACK". O cliente envia uma string contendo o nome do diretório desejado. O servidor processa a requisição, lista o conteúdo do diretório e envia de volta a lista de nomes de arquivos. O cliente encerra a sessão enviando "bye" e calcula o tempo total gasto na comunicação. A fase mais crítica do trabalho é a avaliação do desempenho, então foi implementada uma metodologia para medir o throughput (taxa de transferência) da comunicação, definida como o número total de bytes enviados dividido pelo tempo medido no cliente.

Metodologia

Inicialmente, ao tentar estabelecer conexão, são requisitados três argumentos: <host do servidor>, <porto servidor> e <nome diretório> - os dois primeiros, apenas, ao iniciar client e todos os três em server. É neste primeiro argumento que recebe-se o endereço de IP e, como dito, seria necessário que o programa funcionasse para ambos protocolos IPv4 e IPv6.

Para tal, no servidor, primeiramente, cria-se o socket configurado explicitamente para a família IPv6 (socket.AF_INET6), mas, em seguida, faz-se uma ativação do dual-stack ao desabilitar a opção IPV6_V6ONLY - ao defini-la como 0. Ademais, faz-se um binding genérico no endereço coringa IPv6 se o host fornecido for localhost ou 0.0.0.0. Estas configurações permitem que o socket aceite conexões feitas por ambas famílias de IP no mesmo ponto de escuta. Já no lado do cliente, utiliza-se uma abordagem iterativa baseada na função socket.getaddrinfo(); onde inicia-se consultando o sistema operacional e garantindo que a função retorne todos os endereços possíveis para <host do servidor> e <porto servidor> fornecidos. Assim o código itera sobre a lista de endereços obtidos, adaptando-se a cada um com os devidos protocolos e tenta-se a conexão. A iteração é interrompida no primeiro endereço que resultar em uma conexão bem sucedida, seja ele IPv6 ou IPv4.

Outro dos objetivos era observar o desempenho dos programas nas trocas de mensagem, através da medição do throughput da comunicação. Assim, conduziu-se uma bateria de testes com diretórios contendo diferentes tamanhos de mensagem (tamanho da lista de nomes dos arquivos) para observar o comportamento dos índices. Os experimentos foram realizados para verificar como o desempenho varia em função do tamanho da mensagem, incluindo medições para tamanhos que variam em potências de 2^i bytes, com $0 \leq i \leq 16$; e para cada um dos tamanhos de mensagem, foram feitas 10 rodagens do programa - utilizando um script auxiliar -, a fim de se ter uma quantidade de dados interessante para medidas como tempo médio, variância e desvio

padrão. Os tempos foram coletados utilizando a função `time.perf_counter()` da biblioteca `time` do `python`; função esta que mede com a precisão desejada o tempo de execução durante o envio de mensagens.

Os programas funcionaram e tiveram sua conexão estabelecida rodando no mesmo computador, na configuração de `localhost` enquanto testava-se sua funcionalidade inicialmente. O computador utilizado para esse processo de construção e rodagem de todos testes, por sua vez, possui processador `AMD Ryzen 7 2700` - com 8 cores e 3.20 GHz -, 24 GB de memória principal - `DDR4` e 2133 MHz -, GPU `NVIDIA GeForce RTX 2060` - 6 GB - e sistema operacional `Windows 10`.

Resultados

Conforme as diretrizes descritas na Metodologia, os testes foram executados e separados pelo protocolo IP. Sem falhas na execução, pôde-se ver então que os programas estão suficientemente preparados para tanto `IPv6` quanto `IPv4`. Os resultados foram conforme apresentados nas Tabelas 1 e 2 e na Figura 1.

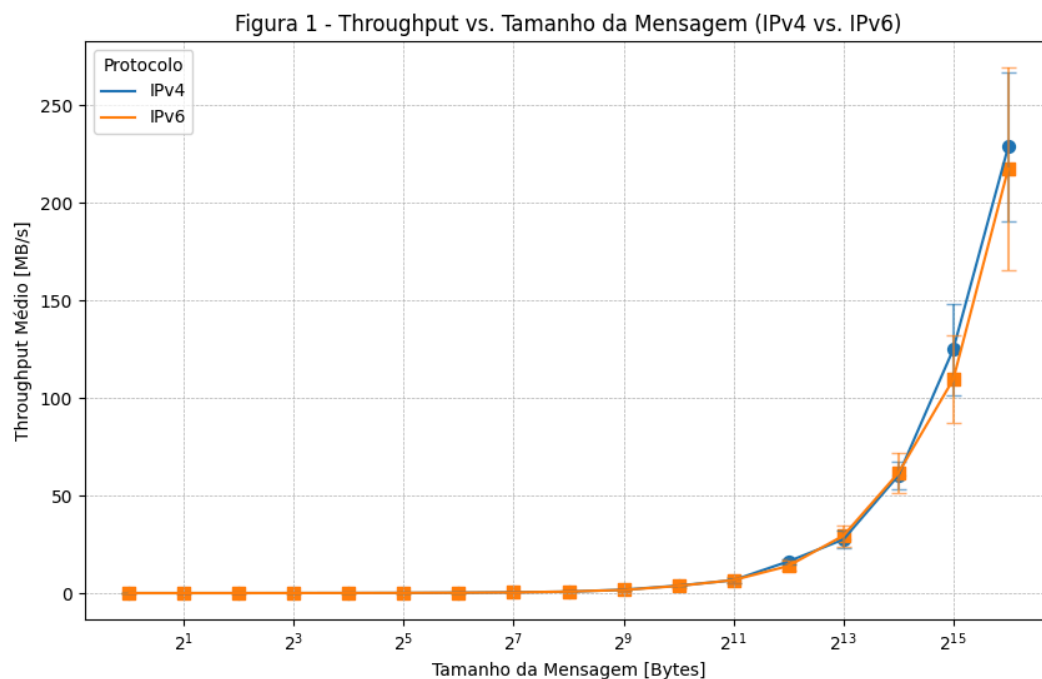
Tabela 1 - Resultados das medições de throughput (`IPv4`)

Tamanho (Bytes)	Nº Mensagens	Tempo Médio (s)	Desvio Padrão (s)	Banda Média (KB/s)
1	1	0.000242	0.000006	4.03
2	1	0.000317	0.000009	6.17
4	1	0.000301	0.000042	12.98
8	1	0.000269	0.000037	29.06
16	1	0.000306	0.000062	51.15
32	1	0.00031	0.000043	100.81
64	1	0.00029	0.000044	215.37
128	1	0.000284	0.000007	440.92
256	1	0.000327	0.000151	764.29
512	1	0.000284	0.000049	1758.71
1024	1	0.000255	0.000002	3927.73
2048	1	0.000295	0.000054	6779.66
4096	1	0.00024	0.000018	16632.00
8192	1	0.000284	0.000044	28159.10
16384	1	0.00026	0.000031	61633.30
32768	1	0.00025	0.000047	127949.00
65536	1	0.000273	0.000046	234175.00

Tabela 2 - Resultados das medições de throughput (`IPv6`)

Tamanho (Bytes)	Nº Mensagens	Tempo Médio (s)	Desvio Padrão (s)	Banda Média (KB/s)
1	1	0.00032	0.000087	3.06
2	1	0.000323	0.000074	6.05
4	1	0.000288	0.000036	13.58

8	1	0.000303	0.000035	25.78
16	1	0.000319	0.000069	49.00
32	1	0.000289	0.000053	108.21
64	1	0.000285	0.000055	219.14
128	1	0.000331	0.000092	378.10
256	1	0.000308	0.000072	812.74
512	1	0.000298	0.000081	1676.16
1024	1	0.000265	0.000064	3777.86
2048	1	0.000293	0.000062	6830.60
4096	1	0.000281	0.000051	14250.10
8192	1	0.000265	0.000049	30143.20
16384	1	0.000254	0.000043	62992.10
32768	1	0.000285	0.000058	112163.00
65536	1	0.000287	0.000069	222686.00



Análise

Os resultados dos testes apresentaram um comportamento, de certa forma, já esperado, apresentando valores que mostram uma operação extremamente eficiente, característica de uma comunicação em um mesmo dispositivo. Conforme evidenciado nas medições, o throughput apresentou uma forte dependência do tamanho da mensagem - como também já esperado. No entanto, é interessante notar o comportamento da

curva na Figura 1 até valores de tamanho de mensagem igual 2^9 , onde os valores do throughput são muito próximos apesar do crescimento exponencial da quantidade de bytes.

É somente na segunda metade do experimento, isto é para tamanhos de mensagem de $9 \leq i \leq 16$, em que observa-se um crescimento acelerado e exponencial dos valores de saída, acompanhado de maiores discrepâncias na variância. Fazendo surgir dúvidas sobre o comportamento dessas saídas para valores além da escala proposta; dúvidas estas que não serão respondidas por este experimento.

Em relação aos protocolos de rede, a comparação direta entre IPv4 e IPv6 demonstrou que as curvas de throughput e as médias de tempo estão praticamente sobrepostas em toda a faixa de medição - com uma leve superioridade numérica em throughput para IPv4 perto do fim da escala de tamanhos de mensagem. Isso comprova que a implementação dual-stack e a eficiência do socket do sistema operacional são consistentes, sem penalidade de desempenho significativa ao utilizar o endereçamento IPv6.

Conclusão

Em suma, o sistema com o par de programas foi implementado com sucesso e cumpriu os objetivos que foram propostos. A metodologia de medição de throughput, baseada em múltiplas repetições e análise estatística, forneceu uma visão detalhada da eficiência do sistema, demonstrando a clara relação entre o tamanho da carga útil e a taxa de transferência. Os resultados não apenas validam a funcionalidade e a estabilidade da aplicação, como também confirmam que a arquitetura dual-stack funciona de forma transparente e eficiente.

Ademais, produzir esse experimento é de incalculável valor para o aprendizado dos tópicos explorados em relação ao conhecimento em redes de computadores. A aplicação prática dos conteúdos permite visualizar de forma incomparável o funcionamento destes, trazendo toda uma nova camada de compreensão; além de tudo é uma experiência agradável e divertida.